AutoEIT: Automated Transcription of Spanish EIT Learner Audio

This notebook walks through the complete pipeline for transcribing Spanish Elicited Imitation Task (EIT) audio recordings from second language learners.

Pipeline stages:

1. Data exploration — understand the audio files and Excel structure
2. Audio analysis — examine energy patterns and turn-taking structure
3. Transcription — run Whisper ASR with optimized parameters
4. Post-processing — match segments to target sentences
5. Output — write results to Excel
6. Evaluation — assess transcription quality

1. Setup & Data Exploration

In [1]:
```
import whisper
import librosa
import numpy as np
import json
import os
import openpyxl
import Levenshtein

print("All imports successful.")

import torch
device = "cuda" if torch.cuda.is_available() else "cpu"
print(f"Device: {device}")
```

```
All imports successful.
Device: cpu
```

1.1 Explore the Excel Template

The dataset is organized in an Excel file with one tab per participant. Column A has the sentence number (1–30), column B has the target stimulus with syllable count, and column C is where our transcriptions go.

In [2]:
```
wb = openpyxl.load_workbook('data/AutoEIT_Sample_Audio_for_Transcribing.xlsx
print("Sheet names:", wb.sheetnames)

# Show the Info sheet
ws = wb['Info']
print("\n--- INFO ---")
```

```
for row in ws.iter_rows(min_row=1, max_row=ws.max_row, values_only=True):
    vals = [v for v in row if v is not None]
    if vals:
        print(vals[0])
```

```
Sheet names: ['Info', '38010-2A', '38011-1A', '38012-2A', '38015-1A']

--- INFO ---
This file contains 4 sample audio files of learner EITs to be transcribed fo
r subsequent scoring.
Each participant tab contains a link to the audio file needed.
All files are EIT vA (same stimuli)
Stimuli (prompts to be repeated) are listed in column B. The number provided
in parenthesis after the stimuli sentence is the number of syllables.
Participant transcriptions should be entered into column C.
It is very important that participant utterances are transcribed without cor
rections as the evaluation rubric focuses on maintenance of meaning.
Note: The file begins with instructions for the participant (on the recordin
g they hear), followed by practice sentences in English (which do not need t
o be transcribed). Relevant audio begins around 2:30 for most files.
Note: Because participants are repeating sentences, there are long pauses in
the audio files (when the participant is listening to the recording).
```

In [3]:

```
# Show the 30 target sentences
ws = wb['38015-1A']
print("Target sentences (EIT version A):")
print(f"{'#':>2} | Stimulus")
print("-" * 60)
for row in range(2, 32):
    item = ws.cell(row=row, column=1).value
    stimulus = ws.cell(row=row, column=2).value
    print(f"{item:>2} | {stimulus}")
```

```
Target sentences (EIT version A):
 # | Stimulus
------------------------------------------------------------
 1 | Quiero cortarme el pelo (7)
 2 | El libro está en la mesa (7)
 3 | El carro lo tiene Pedro (8)
 4 | El se ducha cada mañana (9)
 5 | ¿Qué dice usted que va a hacer hoy? (9)
 6 | Dudo que sepa manejar muy bien (10)
 7 | Las calles de esta ciudad son muy anchas (11)
 8 | Puede que llueva mañana todo el día (12)
 9 | Las casas son muy bonitas pero caras (12)
10 | Me gustan las películas que acaban bien (12)
11 | El chico con el que yo salgo es español (13)
12 | Después de cenar me fui a dormir tranquilo (13)
13 | Quiero una casa en la que vivan mis animales (14)
14 | A nosotros nos fascinan las fiestas grandiosas (14)
15 | Ella sólo bebe cerveza y no come nada (15)
16 | Me gustaría que el precio de las casas bajara (15)
17 | Cruza a la derecha y después sigue todo recto (15)
18 | Ella ha terminado de pintar su apartamento (15)
19 | Me gustaría que empezara a hacer más calor pronto (15)
20 | El niño al que se le murió el gato está triste (16)
21 | Una amiga mía cuida a los niños de mi vecino (16)
22 | El gato que era negro fue perseguido por el perro (16)
23 | Antes de poder salir él tiene que limpiar su cuarto (16)
24 | La cantidad de personas que fuman ha disminuido (17)
25 | Después de llegar a casa del trabajo tomé la cena (17)
26 | El ladrón al que atrapó la policía era famoso (17)
27 | Le pedí a un amigo que me ayudara con la tarea (17)
28 | El examen no fue tan difícil como me habían dicho (17)
29 | ¿Serías tan amable de darme el libro que está en la mesa? (17)
30 | Hay mucha gente que no toma nada para el desayuno (17)
```

1.2 Explore the Audio Files

Each participant has one continuous audio recording. The structure is:

- **0:00 – ~2:30** — English instructions and practice sentences (skip)
- **~2:30 onward** — 30 EIT items, each with: stimulus → pause → learner response → longer pause

Exception: Participant 038012 starts at 12:00.

In [4]:
```
audio_dir = "data/audio"

for filename in sorted(os.listdir(audio_dir)):
    if not filename.endswith(".mp3"):
        continue
    filepath = os.path.join(audio_dir, filename)
    y, sr = librosa.load(filepath, sr=None)
    duration = librosa.get_duration(y=y, sr=sr)
    minutes = int(duration // 60)
    seconds = int(duration % 60)
```

```
    print(f"\n{filename}")
    print(f"  Duration:    {minutes}:{seconds:02d}")
    print(f"  Sample rate: {sr} Hz")
    print(f"  Channels:    {'mono' if y.ndim == 1 else 'stereo'}")
    print(f"  Peak volume: {abs(y).max():.3f}")
    print(f"  Mean volume: {abs(y).mean():.4f}")
```

```
/Users/david/PycharmProjects/AutoEIT_test/.venv/lib/python3.9/site-packages/
tqdm/auto.py:21: TqdmWarning: IProgress not found. Please update jupyter and
ipywidgets. See https://ipywidgets.readthedocs.io/en/stable/user_install.htm
l
  from .autonotebook import tqdm as notebook_tqdm
038010_EIT-2A.mp3
  Duration:    9:09
  Sample rate: 44100 Hz
  Channels:    mono
  Peak volume: 0.226
  Mean volume: 0.0017

038011_EIT-1A.mp3
  Duration:    9:07
  Sample rate: 44100 Hz
  Channels:    mono
  Peak volume: 1.062
  Mean volume: 0.0050

038012_EIT-2A.mp3
  Duration:    18:45
  Sample rate: 44100 Hz
  Channels:    mono
  Peak volume: 1.032
  Mean volume: 0.0020

038015_EIT-1A.mp3
  Duration:    8:49
  Sample rate: 44100 Hz
  Channels:    mono
  Peak volume: 0.793
  Mean volume: 0.0026
```

Key observations:

- 038012 is 18:45 long (EIT starts at 12:00), while others are ~9 minutes (EIT at ~2:30)
- 038010 has very low volume (peak 0.226) — a quiet recording
- Very low mean volumes across all files — confirms long silent periods between items

2. Audio Analysis

Before transcribing, let's examine the energy patterns to understand the turn-taking structure.

In [5]:
```
# Energy timeline for participant 038015 (our best participant)
y, sr = librosa.load("data/audio/038015_EIT-1A.mp3", sr=16000)

print("Energy timeline (2:20 - 3:20, first 5 EIT items)")
print("Alternating pattern: STIMULUS (loud) → pause → LEARNER (quieter) → lo
print("=" * 60)
for t in range(140, 200):
    chunk = y[t * sr:(t + 1) * sr]
    rms = np.sqrt(np.mean(chunk ** 2))
    bar = "▇" * int(rms * 500)
    print(f"  {t//60}:{t%60:02d} | {rms:.4f} | {bar}")
```

```
Energy timeline (2:20 - 3:20, first 5 EIT items)
Alternating pattern: STIMULUS (loud) → pause → LEARNER (quieter) → longer pa
use
============================================================
  2:20 | 0.0002 |
  2:21 | 0.0002 |
  2:22 | 0.0002 |
  2:23 | 0.0026 | ▌
  2:24 | 0.0015 |
  2:25 | 0.0016 |
  2:26 | 0.0012 |
  2:27 | 0.0012 |
  2:28 | 0.0089 | ▌▌▌▌
  2:29 | 0.0129 | ▌▌▌▌▌▌
  2:30 | 0.0024 | ▌▌
  2:31 | 0.0014 |
  2:32 | 0.0016 |
  2:33 | 0.0013 |
  2:34 | 0.0011 |
  2:35 | 0.0021 | ▌▌
  2:36 | 0.0242 | ▌▌▌▌▌▌▌▌▌▌▌▌
  2:37 | 0.0074 | ▌▌▌▌
  2:38 | 0.0012 |
  2:39 | 0.0016 |
  2:40 | 0.0012 |
  2:41 | 0.0011 |
  2:42 | 0.0011 |
  2:43 | 0.0222 | ▌▌▌▌▌▌▌▌▌▌▌▌
  2:44 | 0.0109 | ▌▌▌▌▌▌
  2:45 | 0.0099 | ▌▌▌▌▌
  2:46 | 0.0022 | ▌▌
  2:47 | 0.0014 |
  2:48 | 0.0017 |
  2:49 | 0.0015 |
  2:50 | 0.0012 |
  2:51 | 0.0182 | ▌▌▌▌▌▌▌▌▌▌
  2:52 | 0.0225 | ▌▌▌▌▌▌▌▌▌▌▌▌
  2:53 | 0.0015 |
  2:54 | 0.0013 |
  2:55 | 0.0012 |
  2:56 | 0.0012 |
  2:57 | 0.0012 |
  2:58 | 0.0017 |
  2:59 | 0.0019 |
  3:00 | 0.0149 | ▌▌▌▌▌▌▌▌
  3:01 | 0.0180 | ▌▌▌▌▌▌▌▌▌▌
  3:02 | 0.0108 | ▌▌▌▌▌▌
  3:03 | 0.0013 |
  3:04 | 0.0011 |
  3:05 | 0.0011 |
  3:06 | 0.0012 |
  3:07 | 0.0028 | ▌▌
  3:08 | 0.0016 |
  3:09 | 0.0112 | ▌▌▌▌▌▌
  3:10 | 0.0172 | ▌▌▌▌▌▌▌▌▌▌
  3:11 | 0.0043 | ▌▌▌
```

```
3:12 | 0.0013 |
3:13 | 0.0023 | ▉
3:14 | 0.0023 | ▉
3:15 | 0.0013 |
3:16 | 0.0015 |
3:17 | 0.0013 |
3:18 | 0.0014 |
3:19 | 0.0201 | ▉▉▉▉▉▉▉▉
```

The alternating pattern is clear: spikes of speech separated by silence. Each EIT item consists of a stimulus burst followed shortly by a learner response burst.

2.1 Detect Speech Bursts and Group into Items

In [6]:
```
def detect_speech_bursts(y, sr, start_sec, threshold=0.003, min_duration=0.3
    """Detect speech bursts using energy threshold."""
    frame_length = int(0.5 * sr)
    hop_length = int(0.25 * sr)
    rms = librosa.feature.rms(y=y, frame_length=frame_length, hop_length=hop
    times = librosa.frames_to_time(range(len(rms)), sr=sr, hop_length=hop_le

    is_speech = rms > threshold
    bursts = []
    in_burst = False
    start = 0
    for i, speaking in enumerate(is_speech):
        if speaking and not in_burst:
            start = times[i]
            in_burst = True
        elif not speaking and in_burst:
            end = times[i]
            if end - start > min_duration:
                bursts.append({"start": round(start, 2), "end": round(end, 2
            in_burst = False
    return [b for b in bursts if b["start"] >= start_sec]


def group_into_items(bursts, gap_threshold=4.0):
    """Group speech bursts into EIT items based on silence gaps."""
    items = []
    current_item = []
    prev_end = bursts[0]["start"] if bursts else 0
    for b in bursts:
        gap = b["start"] - prev_end
        if gap > gap_threshold and current_item:
            items.append(current_item)
            current_item = [b]
        else:
            current_item.append(b)
        prev_end = b["end"]
    if current_item:
        items.append(current_item)
    return items
```

```
# Analyze all participants
files_info = {
    "038010": ("data/audio/038010_EIT-2A.mp3", 150),
    "038011": ("data/audio/038011_EIT-1A.mp3", 150),
    "038012": ("data/audio/038012_EIT-2A.mp3", 720),
    "038015": ("data/audio/038015_EIT-1A.mp3", 150),
}

for pid, (filepath, skip) in files_info.items():
    y, sr = librosa.load(filepath, sr=16000)
    bursts = detect_speech_bursts(y, sr, start_sec=skip)
    items = group_into_items(bursts)
    no_response = sum(1 for item in items if len(item) == 1)
    print(f"{pid}: {len(items)} items detected | {no_response} with no learr
```

```
038010: 30 items detected | 16 with no learner response detected
038011: 27 items detected | 8 with no learner response detected
038012: 29 items detected | 9 with no learner response detected
038015: 29 items detected | 16 with no learner response detected
```

Key finding: Simple energy-based segmentation is insufficient. Many learner responses are too quiet to detect as separate bursts, especially for 038015 (16 apparent "stimulus only" items). Whisper's neural approach picks up these quiet responses that raw energy analysis misses. This motivated our decision to rely on Whisper for transcription rather than pre-segmenting the audio.

3. Transcription with Whisper

3.1 Whisper Configuration

Key parameter choices:

- `language="es"` — Force Spanish to prevent language-switching on accented speech
- `condition_on_previous_text=False` — Prevents hallucinations from cascading across segments
- `no_speech_threshold=0.6` — Aggressively filters segments where model detects no speech
- `logprob_threshold=-0.5` — Drops low-confidence segments (gibberish/hallucinations)
- `compression_ratio_threshold=2.0` — Catches repetitive hallucinated text

In [7]:
```
# Load model (this downloads ~1.4GB on first run)
MODEL_SIZE = "large-v3"  # Use "medium" for faster iteration on CPU
model = whisper.load_model(MODEL_SIZE)
print(f"Loaded {MODEL_SIZE} model.")
```

```
Loaded large-v3 model.
```

In [8]:
```
def transcribe_audio(model, filepath, skip_seconds):
    """Transcribe audio, returning only EIT segments after skip time."""
    result = model.transcribe(
        filepath,
        language="es",
        task="transcribe",
        no_speech_threshold=0.6,
        logprob_threshold=-0.5,
        compression_ratio_threshold=2.0,
        condition_on_previous_text=False,
    )
    segments = [
        {
            "start": round(s["start"], 2),
            "end": round(s["end"], 2),
            "text": s["text"].strip(),
            "no_speech_prob": round(s["no_speech_prob"], 3),
            "avg_logprob": round(s["avg_logprob"], 3),
        }
        for s in result["segments"]
        if s["start"] >= skip_seconds
    ]
    return segments
```

3.2 Transcribe All Participants

In [9]:
```
AUDIO_FILES = {
    "038010": "data/audio/038010_EIT-2A.mp3",
    "038011": "data/audio/038011_EIT-1A.mp3",
    "038012": "data/audio/038012_EIT-2A.mp3",
    "038015": "data/audio/038015_EIT-1A.mp3",
}

SKIP_SECONDS = {
    "038010": 150,
    "038011": 150,
    "038012": 720,
    "038015": 150,
}

os.makedirs("output", exist_ok=True)
all_segments = {}

for pid, filepath in AUDIO_FILES.items():
    print(f"Transcribing {pid}...")
    segments = transcribe_audio(model, filepath, SKIP_SECONDS[pid])
    all_segments[pid] = segments

    # Save raw output
    with open(f"output/{pid}_raw.json", "w", encoding="utf-8") as f:
        json.dump(segments, f, ensure_ascii=False, indent=2)

    print(f"  {len(segments)} segments")
    for s in segments[:5]:
```

```
        print(f"    [{s['start']:>7.1f} - {s['end']:>7.1f}] {s['text']}")
    if len(segments) > 5:
        print(f"    ... and {len(segments) - 5} more")
    print()
```

```
Transcribing 038010...
/Users/david/PycharmProjects/AutoEIT_test/.venv/lib/python3.9/site-packages/
whisper/transcribe.py:132: UserWarning: FP16 is not supported on CPU; using
FP32 instead
  warnings.warn("FP16 is not supported on CPU; using FP32 instead")
  42 segments
    [  171.0 -   174.1] El libro está en la mesa.
    [  176.1 -   177.4] El carro no tiene pelo.
    [  179.4 -   181.7] El carro no tiene pelo.
    [  183.8 -   185.4] Él se ducha cada mañana.
    [  187.5 -   189.8] Él se ducha cada mañana.
    ... and 37 more

Transcribing 038011...
/Users/david/PycharmProjects/AutoEIT_test/.venv/lib/python3.9/site-packages/
whisper/transcribe.py:132: UserWarning: FP16 is not supported on CPU; using
FP32 instead
  warnings.warn("FP16 is not supported on CPU; using FP32 instead")
  39 segments
    [  150.0 -   153.0] Quiero cortarme el pelo.
    [  157.0 -   160.0] El libro está en la mesa.
    [  165.0 -   168.0] El carro no tiene pelo.
    [  173.0 -   176.0] Él se ducha cada mañana.
    [  180.0 -   187.0] ¿Qué dice usted que va a hacer hoy?
    ... and 34 more

Transcribing 038012...
/Users/david/PycharmProjects/AutoEIT_test/.venv/lib/python3.9/site-packages/
whisper/transcribe.py:132: UserWarning: FP16 is not supported on CPU; using
FP32 instead
  warnings.warn("FP16 is not supported on CPU; using FP32 instead")
  27 segments
    [  746.3 -   750.3] El libro está en la mesa.
    [  750.3 -   758.3] El carro lo tiene a pelo.
    [  758.3 -   766.3] Él se duche cada mañana.
    [  796.3 -   798.3] Los calla santo.
    [  803.3 -   806.3] El pueblo se que ella calla.
    ... and 22 more

Transcribing 038015...
/Users/david/PycharmProjects/AutoEIT_test/.venv/lib/python3.9/site-packages/
whisper/transcribe.py:132: UserWarning: FP16 is not supported on CPU; using
FP32 instead
  warnings.warn("FP16 is not supported on CPU; using FP32 instead")
```

```
29 segments
  [  150.0 -   157.0] El libro está en la mesa.
  [  157.0 -   165.0] El carro no tiene pelo.
  [  165.0 -   173.0] Él se ducha cada mañana.
  [  173.0 -   182.5] ¿Qué dice usted que va a ser hoy?
  [  182.5 -   182.5] ¿Qué dice usted que va a ser hoy?
  ... and 24 more
```

3.3 Why Not Just Use Raw Whisper Output?

Whisper's output has several issues that require post-processing:

1. **Variable segment count** — We get 28–42 segments per file, not the expected 30
2. **Merged items** — Sometimes two EIT items end up in one segment
3. **Split items** — One item can be split across multiple segments
4. **Stimulus capture** — Whisper may transcribe the stimulus instead of the learner
5. **Hallucinations** — Nonsense text during silence (mostly filtered, but not eliminated)

4. Post-Processing: Segment-to-Target Matching

We use normalized Levenshtein similarity to match each transcribed segment to one of the 30 target sentences, then apply greedy assignment.

In [10]:
```
TARGETS = [
    "Quiero cortarme el pelo",
    "El libro está en la mesa",
    "El carro lo tiene Pedro",
    "El se ducha cada mañana",
    "Qué dice usted que va a hacer hoy",
    "Dudo que sepa manejar muy bien",
    "Las calles de esta ciudad son muy anchas",
    "Puede que llueva mañana todo el día",
    "Las casas son muy bonitas pero caras",
    "Me gustan las películas que acaban bien",
    "El chico con el que yo salgo es español",
    "Después de cenar me fui a dormir tranquilo",
    "Quiero una casa en la que vivan mis animales",
    "A nosotros nos fascinan las fiestas grandiosas",
    "Ella sólo bebe cerveza y no come nada",
    "Me gustaría que el precio de las casas bajara",
    "Cruza a la derecha y después sigue todo recto",
    "Ella ha terminado de pintar su apartamento",
    "Me gustaría que empezara a hacer más calor pronto",
    "El niño al que se le murió el gato está triste",
    "Una amiga mía cuida a los niños de mi vecino",
    "El gato que era negro fue perseguido por el perro",
    "Antes de poder salir él tiene que limpiar su cuarto",
    "La cantidad de personas que fuman ha disminuido",
    "Después de llegar a casa del trabajo tomé la cena",
    "El ladrón al que atrapó la policía era famoso",
```

```
    "Le pedí a un amigo que me ayudara con la tarea",
    "El examen no fue tan difícil como me habían dicho",
    "Serías tan amable de darme el libro que está en la mesa",
    "Hay mucha gente que no toma nada para el desayuno",
]


def clean_text(text):
    """Normalize text for comparison."""
    return (text.lower()
            .replace("¿", "").replace("?", "")
            .replace(".", "").replace(",", "")
            .replace("…", "").replace("...", "")
            .strip())


def similarity(a, b):
    """Normalized Levenshtein similarity (1.0 = identical)."""
    a, b = clean_text(a), clean_text(b)
    if not a or not b:
        return 0
    return 1 - Levenshtein.distance(a, b) / max(len(a), len(b))


def match_segments_to_targets(segments, min_similarity=0.25):
    """Greedy best-match assignment of segments to targets."""
    # Score every segment against every target
    scores = []
    for seg in segments:
        for t_idx, target in enumerate(TARGETS):
            sim = similarity(seg["text"], target)
            scores.append((sim, t_idx, seg))

    scores.sort(key=lambda x: x[0], reverse=True)

    # Greedy assignment: each target gets at most one segment
    assigned = {}
    used_segments = set()

    for sim, t_idx, seg in scores:
        seg_key = (seg["start"], seg["end"])
        if sim < min_similarity:
            continue
        if seg_key in used_segments:
            continue
        if t_idx not in assigned:
            assigned[t_idx] = (sim, seg)
            used_segments.add(seg_key)

    # Build results list
    results = []
    for t_idx in range(30):
        if t_idx in assigned:
            sim, seg = assigned[t_idx]
            results.append({
                "item": t_idx + 1,
```

```
                "target": TARGETS[t_idx],
                "transcription": seg["text"],
                "similarity": round(sim, 3),
            })
        else:
            results.append({
                "item": t_idx + 1,
                "target": TARGETS[t_idx],
                "transcription": "[no response detected]",
                "similarity": 0,
            })
    return results

print("Matching functions defined.")
```

```
Matching functions defined.
```

In [11]:

```
# Match all participants
all_results = {}

for pid, segments in all_segments.items():
    results = match_segments_to_targets(segments)
    all_results[pid] = results

    matched = sum(1 for r in results if r["transcription"] != "[no response
    avg_sim = np.mean([r["similarity"] for r in results if r["similarity"] >

    print(f"\n{'='*70}")
    print(f"PARTICIPANT {pid}: {len(segments)} segments → {matched}/30 match
    print(f"{'='*70}")

    for r in results:
        sim = r['similarity']
        text = r['transcription']
        if text == "[no response detected]":
            flag = "← MISSING"
        elif sim >= 0.95:
            flag = "← perfect match"
        elif sim < 0.50:
            flag = "← low confidence"
        else:
            flag = ""
        print(f"  {r['item']:>2}. [{sim:.2f}] {text} {flag}")
```

```
======================================================================
PARTICIPANT 038010: 42 segments → 30/30 matched (avg sim: 0.87)
======================================================================
   1. [0.43] El carro no tiene pelo. ← low confidence
   2. [1.00] El libro está en la mesa. ← perfect match
   3. [0.87] El carro no tiene pelo.
   4. [0.96] Él se ducha cada mañana. ← perfect match
   5. [1.00] ¿Qué dice usted que va a hacer hoy? ← perfect match
   6. [1.00] Dudo que sepa manejar muy bien. ← perfect match
   7. [1.00] Las calles de esta ciudad son muy anchas. ← perfect match
   8. [1.00] Puede que llueva mañana todo el día. ← perfect match
   9. [1.00] Las casas son muy bonitas pero caras. ← perfect match
  10. [1.00] Me gustan las películas que acaban bien. ← perfect match
  11. [1.00] El chico con el que yo salgo es español. ← perfect match
  12. [1.00] Después de cenar, me fui a dormir tranquilo. ← perfect match
  13. [0.91] Quiero una casa en la que vivan animales.
  14. [1.00] A nosotros nos fascinan las fiestas grandiosas. ← perfect match
  15. [0.97] Ella solo bebe cerveza y no come nada. ← perfect match
  16. [0.31] Él se ducha cada mañana. ← low confidence
  17. [0.84] Cruza a la derecha y después siga recto.
  18. [1.00] Ella ha terminado de pintar su apartamento. ← perfect match
  19. [1.00] Me gustaría que empezara a hacer más calor pronto. ← perfect ma
tch
  20. [1.00] El niño al que se le murió el gato está triste. ← perfect match
  21. [0.96] Una amiga mía cuida de los niños de mi vecino. ← perfect match
  22. [0.43] El gato que era negro. ← low confidence
  23. [0.82] Después de poder salir, tiene que limpiar su cuarto.
  24. [0.70] la cantidad de personas que fuman.
  25. [0.41] Después de cenar, me fui a dormir tranquilo. ← low confidence
  26. [0.51] Antes de llegar a casa, el ladrón policía era famoso.
  27. [1.00] Le pedí a un amigo que me ayudara con la tarea. ← perfect match
  28. [1.00] ¿El examen no fue tan difícil como me habían dicho? ← perfect m
atch
  29. [1.00] ¿Serías tan amable de darme el libro que está en la mesa? ← per
fect match
  30. [1.00] Hay mucha gente que no toma nada para el desayuno. ← perfect ma
tch

======================================================================
PARTICIPANT 038011: 39 segments → 30/30 matched (avg sim: 0.80)
======================================================================
   1. [1.00] Quiero cortarme el pelo. ← perfect match
   2. [1.00] El libro está en la mesa. ← perfect match
   3. [0.87] El carro no tiene pelo.
   4. [0.96] Él se ducha cada mañana. ← perfect match
   5. [1.00] ¿Qué dice usted que va a hacer hoy? ← perfect match
   6. [0.85] Todo lo que sepa manejar muy bien.
   7. [1.00] Las calles de esta ciudad son muy anchas. ← perfect match
   8. [1.00] Puede que llueva mañana todo el día. ← perfect match
   9. [1.00] Las casas son muy bonitas pero caras. ← perfect match
  10. [1.00] Me gustan las películas que acaban bien. ← perfect match
  11. [1.00] El chico con el que yo salgo es español. ← perfect match
  12. [1.00] Después de cenar me fui a dormir tranquilo. ← perfect match
  13. [0.80] Quiero una casa que viven a mis animales.
  14. [0.46] a nosotros que las vistas ← low confidence
  15. [0.51] ellas solo beben cerveza
```

```
  16. [0.70] Me gustaría los precios de las caras para nada.
  17. [0.67] Cruza a la derecha y siga derecho.
  18. [0.71] Ella ha terminado los apartamentos.
  19. [1.00] Me gustaría que empezara a hacer más calor pronto. ← perfect ma
tch
  20. [1.00] El niño al que se le murió el gato está triste. ← perfect match
  21. [0.82] Una amiga bien cuidar los niños a mi vecino.
  22. [0.55] El gato era negro y el perro.
  23. [0.41] Parece que salir fue de cuarto. ← low confidence
  24. [0.92] La cantidad de las personas que fuman ha disminuido.
  25. [0.65] Después de llegar a la casa, fui a la cena.
  26. [0.58] Perdón, a la policía eran famoso.
  27. [0.89] Le pedí a un amigo que ayudara con mi tarea.
  28. [0.33] Me gustan las películas que acaban bien. ← low confidence
  29. [0.38] Seria amable que lo estan ← low confidence
  30. [0.84] Hay mucha gente que no tomara el desayuno

======================================================================
PARTICIPANT 038012: 27 segments → 27/30 matched (avg sim: 0.51)
======================================================================
   1. [0.30] qui из-ste libra ← low confidence
   2. [1.00] El libro está en la mesa. ← perfect match
   3. [0.83] El carro lo tiene a pelo.
   4. [0.91] Él se duche cada mañana.
   5. [0.30] Me siento no voy a superpower. ← low confidence
   6. [0.00] [no response detected] ← MISSING
   7. [0.30] Los calla santo. ← low confidence
   8. [0.26] Vuelvo a punto de entender. ← low confidence
   9. [0.89] Las casas son bonitas pero caras.
  10. [0.80] Me gustan las películas pegar bien.
  11. [0.54] El chico es mono español.
  12. [0.45] Después de llegar dormía. ← low confidence
  13. [0.68] Quiero la casa esta a mi animales.
  14. [0.54] A nosotros fascinabas a los nosotros.
  15. [0.29] Ambassador Myrtle Perkins es el conversino de donde estamos. ←
low confidence
  16. [0.55] Me gustaría que los huesos bestas cruzaran a la derecha.
  17. [0.29] Una chica picarettes pisástico. ← low confidence
  18. [0.48] He terminado el fragmento. ← low confidence
  19. [0.51] Me gustaría hacer pintadas por esta, muy pronto.
  20. [0.56] El niño.. El niño Serio Cato es muy triste.
  21. [0.57] Una amiga picandro en mi vecino...
  22. [0.41] El gato tenga nemi, i la perla. ← low confidence
  23. [0.00] [no response detected] ← MISSING
  24. [0.00] [no response detected] ← MISSING
  25. [0.65] Después de llegar a mi casa, voy a la cena.
  26. [0.33] El pueblo se que ella calla. ← low confidence
  27. [0.36] Una estaban viendo del software cuando decían con la cara así .
← low confidence
  28. [0.39] El tarea esta no difícil pero... ← low confidence
  29. [0.27] están en la mesa ← low confidence
  30. [0.31] Hay muchos ejemplos, pero... ← low confidence

======================================================================
PARTICIPANT 038015: 29 segments → 29/30 matched (avg sim: 0.80)
======================================================================
```

```
  1. [0.26] ¿Qué dice usted que va a ser hoy? ← low confidence
  2. [1.00] El libro está en la mesa. ← perfect match
  3. [0.87] El carro no tiene pelo.
  4. [0.96] Él se ducha cada mañana. ← perfect match
  5. [0.91] ¿Qué dice usted que va a ser hoy?
  6. [0.73] ¿Tuvo que sepamanajar bien?
  7. [1.00] Las calles de esta ciudad son muy anchas. ← perfect match
  8. [0.94] Puede que lleve mañana todo el día.
  9. [0.97] Las casas son muy bonitos pero caras. ← perfect match
 10. [1.00] Me gustan las películas que acaban bien. ← perfect match
 11. [0.80] El chico con esto algo es español.
 12. [1.00] Después de cenar, me fui a dormir tranquilo. ← perfect match
 13. [1.00] Quiero una casa en la que vivan mis animales. ← perfect match
 14. [0.91] A nosotros nos fascinan las fiestas grandes.
 15. [0.89] Ella solo bebe cerveza y come nada.
 16. [0.58] Me gustaría las preciosas cajas.
 17. [0.73] Cruza a la derecha y sigue el todo resto.
 18. [0.00] [no response detected] ← MISSING
 19. [0.49] ha terminado de pintar su apartamento. Me gustaría hacer más ca
lor pronto. ← low confidence
 20. [0.80] El niño que se murió al gato y está triste.
 21. [0.61] Una amiga me cuidaba a los niños.
 22. [0.80] El gato negro fuera perseguido por el perro.
 23. [0.59] Antes de salir limpiar el cuarto.
 24. [0.74] La cantidad de personas que fuman es...
 25. [0.78] Después de llegar a la casa después del trabajo come la cena.
 26. [0.73] El ladrón que atrapó con la publicidad era famosa.
 27. [0.89] Le pedí a un amigo que me ayude en la tarea.
 28. [0.59] La prueba no fue difícil, me han dicho.
 29. [0.69] Si eres tan amable, puedes darme el libro en la mesa.
 30. [1.00] Hay mucha gente que no toma nada para el desayuno. ← perfect ma
tch
```

5. Write Results to Excel

In [12]:

```python
SHEET_NAMES = {
    "038010": "38010-2A",
    "038011": "38011-1A",
    "038012": "38012-2A",
    "038015": "38015-1A",
}

template_path = "data/AutoEIT_Sample_Audio_for_Transcribing.xlsx"
output_path = "output/AutoEIT_Transcriptions_Complete.xlsx"

wb = openpyxl.load_workbook(template_path)

for pid, results in all_results.items():
    ws = wb[SHEET_NAMES[pid]]
    for r in results:
        row = r["item"] + 1
        text = r["transcription"]
        if text == "[no response detected]":
            text = "[no response detected]"
        ws.cell(row=row, column=3, value=text)
```

```
    print(f"{pid}: Wrote 30 transcriptions to sheet {SHEET_NAMES[pid]}")

wb.save(output_path)
print(f"\nSaved to {output_path}")
```

```
038010: Wrote 30 transcriptions to sheet 38010-2A
038011: Wrote 30 transcriptions to sheet 38011-1A
038012: Wrote 30 transcriptions to sheet 38012-2A
038015: Wrote 30 transcriptions to sheet 38015-1A

Saved to output/AutoEIT_Transcriptions_Complete.xlsx
```

6. Evaluation

6.1 Summary Statistics

In [13]:
```
print(f"{'Participant':>12} | {'Segments':>8} | {'Matched':>8} | {'Avg Sim':
print("-" * 85)

for pid, results in all_results.items():
    segments = len(all_segments[pid])
    matched = sum(1 for r in results if r["transcription"] != "[no response
    sims = [r["similarity"] for r in results if r["similarity"] > 0]
    avg_sim = np.mean(sims) if sims else 0
    perfect = sum(1 for r in results if r["similarity"] >= 0.95)
    low_conf = sum(1 for r in results if 0 < r["similarity"] < 0.50)
    missing = sum(1 for r in results if r["transcription"] == "[no response

    print(f"{pid:>12} | {segments:>8} | {matched:>7}/30 | {avg_sim:>7.2f} |
```

```
 Participant | Segments |  Matched |  Avg Sim |  Perfect |  Low Conf |  Miss
ing
-------------------------------------------------------------------------
---------
      038010 |       42 |      30/30 |    0.87 |       20 |         4 |
0
      038011 |       39 |      30/30 |    0.80 |       12 |         4 |
0
      038012 |       27 |      27/30 |    0.51 |        1 |        14 |
3
      038015 |       29 |      29/30 |    0.80 |        8 |         2 |
1
```

6.2 Error Analysis

Let's examine the types of errors across participants.

In [14]:
```
# Categorize results
for pid, results in all_results.items():
    categories = {"perfect_match": 0, "learner_error": 0, "low_confidence":

    for r in results:
        sim = r["similarity"]
        if r["transcription"] == "[no response detected]":
```

```
            categories["missing"] += 1
        elif sim >= 0.95:
            categories["perfect_match"] += 1
        elif sim >= 0.50:
            categories["learner_error"] += 1
        else:
            categories["low_confidence"] += 1

    print(f"\n{pid}:")
    print(f"  Perfect match (≥0.95): {categories['perfect_match']:>3}  (lear
    print(f"  Learner errors (0.50+): {categories['learner_error']:>3}  (cle
    print(f"  Low confidence (<0.50): {categories['low_confidence']:>3}  (pc
    print(f"  Missing:                {categories['missing']:>3}  (no segmer
```

```
038010:
  Perfect match (≥0.95):  20  (learner repeated correctly OR stimulus captur
ed)
  Learner errors (0.50+):   6  (clear learner production with errors — ideal
output)
  Low confidence (<0.50):   4  (possible hallucination or very low proficien
cy)
  Missing:                  0  (no segment matched)

038011:
  Perfect match (≥0.95):  12  (learner repeated correctly OR stimulus captur
ed)
  Learner errors (0.50+):  14  (clear learner production with errors — ideal
output)
  Low confidence (<0.50):   4  (possible hallucination or very low proficien
cy)
  Missing:                  0  (no segment matched)

038012:
  Perfect match (≥0.95):   1  (learner repeated correctly OR stimulus captur
ed)
  Learner errors (0.50+):  12  (clear learner production with errors — ideal
output)
  Low confidence (<0.50):  14  (possible hallucination or very low proficien
cy)
  Missing:                  3  (no segment matched)

038015:
  Perfect match (≥0.95):   8  (learner repeated correctly OR stimulus captur
ed)
  Learner errors (0.50+):  19  (clear learner production with errors — ideal
output)
  Low confidence (<0.50):   2  (possible hallucination or very low proficien
cy)
  Missing:                  1  (no segment matched)
```

6.3 Examples of Captured Learner Errors

These demonstrate successful transcription — the pipeline captured what the learner actually said, including their errors.

In [15]:
```
# Show the clearest examples of captured learner errors
print("Examples of successfully captured learner errors:")
print("=" * 80)

for pid in ["038015", "038011"]:
    results = all_results[pid]
    interesting = [r for r in results if 0.50 < r["similarity"] < 0.90]
    for r in interesting[:5]:
        print(f"\n  [{pid}] Item {r['item']} (similarity: {r['similarity']:.
        print(f"  Target:       {r['target']}")
        print(f"  Transcribed:  {r['transcription']}")
```

```
Examples of successfully captured learner errors:
================================================================================
====

  [038015] Item 3 (similarity: 0.87)
  Target:       El carro lo tiene Pedro
  Transcribed:  El carro no tiene pelo.

  [038015] Item 6 (similarity: 0.73)
  Target:       Dudo que sepa manejar muy bien
  Transcribed:  ¿Tuvo que sepamanajar bien?

  [038015] Item 11 (similarity: 0.80)
  Target:       El chico con el que yo salgo es español
  Transcribed:  El chico con esto algo es español.

  [038015] Item 15 (similarity: 0.89)
  Target:       Ella sólo bebe cerveza y no come nada
  Transcribed:  Ella solo bebe cerveza y come nada.

  [038015] Item 16 (similarity: 0.58)
  Target:       Me gustaría que el precio de las casas bajara
  Transcribed:  Me gustaría las preciosas cajas.

  [038011] Item 3 (similarity: 0.87)
  Target:       El carro lo tiene Pedro
  Transcribed:  El carro no tiene pelo.

  [038011] Item 6 (similarity: 0.85)
  Target:       Dudo que sepa manejar muy bien
  Transcribed:  Todo lo que sepa manejar muy bien.

  [038011] Item 13 (similarity: 0.80)
  Target:       Quiero una casa en la que vivan mis animales
  Transcribed:  Quiero una casa que viven a mis animales.

  [038011] Item 15 (similarity: 0.51)
  Target:       Ella sólo bebe cerveza y no come nada
  Transcribed:  ellas solo beben cerveza

  [038011] Item 16 (similarity: 0.70)
  Target:       Me gustaría que el precio de las casas bajara
  Transcribed:  Me gustaría los precios de las caras para nada.
```

7. Challenges & Limitations

Challenge 1: Hallucination During Silence

Without the filtering parameters, Whisper generated repeated "Gracias" during the instruction period and nonsense text (including Korean, Russian characters) during inter-item silence. The `no_speech_threshold`, `logprob_threshold`, and `compression_ratio_threshold` parameters largely solved this.

Challenge 2: Stimulus vs. Learner Separation

Each item contains both the stimulus and the learner's response. Energy-based separation (RMS volume) was unreliable because learner and stimulus volumes overlap. Perfect-match transcriptions (similarity ≥ 0.95) are ambiguous — they may be the stimulus rather than a perfect learner repetition.

Challenge 3: Very Low Proficiency (038012)

This participant had minimal intelligible production. Audio analysis showed no response for ~8 items. Where they did respond, speech was too fragmentary for Whisper to resolve, producing hallucinated output. This is a fundamental ASR limitation.

Challenge 4: Segment Alignment

Whisper occasionally merges two items into one segment or splits one across multiple segments. The matching algorithm handles most cases but occasionally assigns a transcription to the wrong item.

8. Future Improvements

Short-term: Use spectral features (not just RMS) to separate stimulus from learner. Use WhisperX for word-level timestamps. Flag low-confidence items for human review.

Medium-term: Fine-tune Whisper on human-transcribed learner speech data. Run multiple ASR models with majority voting.

Long-term: Build an active learning pipeline that routes uncertain items to human transcribers and uses corrections to improve the model iteratively.